QUIZ 4

Qes1)

```
SELECT *
From Employee;
```

Qes2)

```
SELECT First_Name,Last_Name
From Employee;
```

Qes3)

```
SELECT First_Name AS "Employee Name"
From Employee;
```

Qes4)

```
SELECT UPPER(First_Name)
From Employee;
```

Qes5)

```
SELECT LOWER(First_Name)
From Employee;
```

Qes6)

```
SELECT DISTINCT DEPARTMENT
FROM Employee;
```

Qes7)

```
SELECT SUBSTR(FIRST_NAME, 1, 3)
FROM Employee;
```

Qes8)

```
SELECT INSTR (FIRST_NAME, 'o')
FROM Employee
WHERE FIRST_NAME = 'John';
```

Qes9)

```
SELECT RTRIM(FIRST_NAME)
```

```
FROM Employee;
```

Qes10)

```
SELECT LTRIM(FIRST_NAME)
FROM Employee;
```

Qes11)

```
SELECT LENGTH(FIRST_NAME)
FROM Employee;
```

Qes12)

```
SELECT REPLACE (FIRST_NAME, 'o', '$')
FROM EMPLOYEE;
```

Qes13)

```
SELECT FIRST_NAME || '_' || LAST_NAME
FROM EMPLOYEE;
```

Qes14)

```
SELECT
    FIRST_NAME,
    TO_CHAR(JOINING_DATE, 'YYYY') AS "JOINING YEAR",
    TO_CHAR(JOINING_DATE, 'MM') AS "JOINING MONTH",
    TO_CHAR(JOINING_DATE, 'DD') AS "JOINING DATE",
    JOINING_DATE

FROM EMPLOYEE;
```

Qes15)

```
SELECT *
FROM EMPLOYEE
ORDER BY FIRST_NAME ASC;
```

Qes16)

```
SELECT *
FROM EMPLOYEE
ORDER BY FIRST_NAME DESC;
```

Qes17)

```sql
SELECT *
FROM EMPLOYEE
ORDER BY FIRST_NAME ASC, SALARY DESC;
```

Qes18)

```sql
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME = 'John';
```

Qes19)

```sql
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME IN ('John', 'Roy');
```

OR

```sql
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME = 'John'
   OR FIRST_NAME = 'Roy';
```

Qes20)

```sql
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME NOT IN ('John', 'Roy');
```

Qes21)

```sql
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME LIKE 'J%';
```

Qes22)

```sql
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME LIKE '%o%';
```

Qes23)

```sql
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME LIKE '%n';
```

Qes24)

Wildcard	Meaning	Example

%	Zero or more characters	'J%'
_	Exactly one character	'J___'

-- Starts with J

WHERE FIRST_NAME LIKE 'J%'

-- Contains o

WHERE FIRST_NAME LIKE '%o%'

-- Ends with n

WHERE FIRST_NAME LIKE '%n'

-- Exactly 4 letters

WHERE FIRST_NAME LIKE '____'

-- Starts with J and exactly 4 letters

WHERE FIRST_NAME LIKE 'J___'

Qes25)

```
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME LIKE '___n';
```

Qes26)

```
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME LIKE 'J___';
```

Qes27)

```
SELECT *
FROM EMPLOYEE
WHERE SALARY > 600000;
```

Qes28)

```
SELECT *
FROM EMPLOYEE
WHERE SALARY < 800000;
```

Qes29)

```
SELECT *
FROM EMPLOYEE
WHERE SALARY BETWEEN 500000 AND 800000;
```

Qes30)

```
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME IN ('John', 'Michael');
```

OR

```
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME = 'John'
   OR FIRST_NAME = 'Michael';
```

Qes31)

```
SELECT *
FROM Employee
WHERE EXTRACT (YEAR FROM JOINING_DATE) = 2013;
```

Qes32)

```
SELECT *
FROM Employee
WHERE EXTRACT (MONTH FROM JOINING_DATE) = 1;
```

Qes33)

```
SELECT *
FROM Employee
WHERE JOINING_DATE < DATE '2013-01-01';
```

Qes34)

```
SELECT *
FROM Employee
WHERE JOINING_DATE > DATE '2013-01-31';
```

Qes35)

```
SELECT TO_CHAR(JOINING_DATE, 'DD-MON-YYYY HH24:MI:SS')
FROM Employee;
```

Qes36)

It can’t be returned cause the date datatype cannot store milliseconds

Qes37)

```sql
SELECT
    e.EmployeeId,
    e.FirstName,
    e. JOINING_DATE,
    i.Incentive_Date,
    i.Incentive_Date - e. JOINING_DATE AS Date_Difference
FROM Employee e
JOIN Incentives i
    ON e.EmployeeId = i.IncentiveRefId;
```

Qes38)

```sql
SELECT SYSDATE
FROM dual;
```

Qes39)

```sql
SELECT FirstName, LastName
FROM Employee
WHERE LastName LIKE '%\%%' ESCAPE '\';
```

Qes40)

```sql
SELECT REPLACE(LastName, '%', ' ') AS LastName
FROM Employee;
```

Qes41)

```sql
SELECT Department,
       SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department;
```

Qes42)

```sql
SELECT Department,
       SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department
ORDER BY Total_Salary DESC;
```

Qes43)

```sql
SELECT Department,
       COUNT (*) AS No_Of_Employees,
       SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department
ORDER BY Total_Salary DESC;
```

Qes44)

```sql
SELECT Department,
```

```
       AVG(Salary) AS Average_Salary,Salary
FROM Employee
GROUP BY Department
ORDER BY Salary ASC;
```

Qes45)

```
SELECT Department,
       MAX(Salary) AS Maximum_Salary, Salary
FROM Employee
GROUP BY Department
ORDER BY Salary ASC;
```

Qes46)

```
SELECT Department,
       MIN(Salary) AS Minimum_Salary,Salary
FROM Employee
GROUP BY Department
ORDER BY Salary ASC;
```

Qes47)

```
SELECT COUNT (*) AS No_Of_Employees,
EXTRACT (YEAR FROM JOINING_DATE) AS Joining_Year,
EXTRACT (MONTH FROM JOINING_DATE) AS Joining_Month
FROM Employee
GROUP BY EXTRACT (YEAR FROM JOINING_DATE),
         EXTRACT (MONTH FROM JOINING_DATE)
ORDER BY Joining_Year, Joining_Month;
```

Qes48)

```
SELECT Department,
       SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department
HAVING SUM(Salary) > 800000
ORDER BY Total_Salary DESC;
```

Qes49)

```
SELECT e.FirstName,
       i.IncentiveAmount
FROM Employee e
JOIN Incentives i
    ON e.EmployeeId = i.IncentiveRefId;
```

Qes50)

```
SELECT e.FirstName,
       i.IncentiveAmount
FROM Employee e
JOIN Incentives i
```

```
    ON e.EmployeeId = i.IncentiveRefId
WHERE i.IncentiveAmount > 3000;
```

Qes51)

```
SELECT e.FirstName,
       i.IncentiveAmount
FROM Employee e
LEFT JOIN Incentives i
    ON e.EmployeeId = i.IncentiveRefId;
```

Qes52)

```
SELECT e.FirstName,
       NVL (i.IncentiveAmount, 0) AS IncentiveAmount
FROM Employee e
LEFT JOIN Incentives i
    ON e.EmployeeId = i.IncentiveRefId;
```

Qes53)

```
SELECT e.FirstName,
       i.IncentiveAmount
FROM Employee e
LEFT JOIN Incentives i
    ON e.EmployeeId = i.IncentiveRefId
WHERE i.IncentiveAmount IS NOT NULL;
```

Qes54)

```
SELECT e.FirstName,
       (
           SELECT MAX(i.IncentiveAmount)
           FROM Incentives i
           WHERE i.IncentiveRefId = e.EmployeeId
       ) AS Max_Incentive
FROM Employee e;
```

Qes55)

```
SELECT Salary
FROM Employee
ORDER BY Salary DESC
FETCH FIRST 2 ROWS ONLY;
```

Qes56)

```
SELECT Salary
FROM Employee
ORDER BY Salary DESC
FETCH FIRST N ROWS ONLY;
```

U can specify the N.

Qes57)

```sql
SELECT MAX(Salary) AS Second Highest
FROM Employee
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Employee
);
```

There is another solution using window function like DENSE_RANK () fun

```sql
SELECT Salary
FROM (
    SELECT Salary,
           DENSE_RANK () OVER (ORDER BY Salary DESC) AS rnk
    FROM Employee
)
WHERE rnk = 2;
```

Qes58)

```sql
SELECT Salary
FROM (
    SELECT Salary,
           DENSE_RANK() OVER (ORDER BY Salary DESC) AS rnk
    FROM Employee
)
WHERE rnk = N;
```

U can specify the N

Qes59)

```sql
SELECT FirstName AS Name
FROM Employee

UNION ALL

SELECT LastName AS Name
FROM Employee;
```

Qes60)

UNION:

Combine tables but removes Duplicates Values

UNION ALL:

Combine tables without removing duplicates values

Qes61)

```sql
SELECT *
FROM Employee e
```

```
WHERE EXISTS (
    SELECT 1
    FROM Incentives i
    WHERE i.IncentiveRefId = e.EmployeeId
);
```

Qes62)

Using intersect Operator

```
SELECT EmployeeId
FROM Employee

INTERSECT

SELECT IncentiveRefId
FROM Incentives;
```

Qes63)

```
SELECT e.EmployeeId
FROM Employee e
LEFT JOIN Incentives i
    ON e.EmployeeId = i.IncentiveRefId
WHERE i.IncentiveRefId IS NULL;
```

Qes64)

```
SELECT First_Name,
       Salary,
       CASE
           WHEN First_Name = 'John' THEN Salary * 0.20
           WHEN First_Name = 'Roy'  THEN Salary * 0.10
           ELSE Salary * 0.15
       END AS Salary_Percentage
FROM Employee;
```

Qes65)

```
SELECT First_Name,
       Last_Name, Department
       CASE
           WHEN Department = 'Banking' THEN 'Bank Dept'
           WHEN Department = 'Insurance' THEN 'Insurance Dept'
           WHEN Department = 'Services' THEN 'Services Dept'
       END AS Department
FROM Employee;
```

Qes66)

```
DELETE FROM Employee e
WHERE EXISTS (
    SELECT 1
    FROM Incentives i
    WHERE i.IncentiveRefId = e.EmployeeId
);
```

Qes67)

```
INSERT INTO Employee
    (EmployeeId, FirstName, LastName, Salary, JOINING_DATE, Department)
VALUES
    (9, 'David', 'O''Connor', 70000, DATE '2013-02-01', 'Services');
```

Qes68)

```
SELECT LastName
FROM Employee
WHERE REGEXP_LIKE (LastName, '^[0-9] +$');
```

Qes69)

```
SELECT e.EmployeeId,
       e.FirstName,
       i.IncentiveDate,
       i.IncentiveAmount,
       RANK() OVER (
           PARTITION BY TRUNC(i.IncentiveDate, 'MM')
           ORDER BY i.IncentiveAmount DESC
       ) AS Incentive_Rank
FROM Employee e
JOIN Incentives i
    ON e.EmployeeId = i.IncentiveRefId;
```

Qes70)

```
UPDATE Incentives
SET IncentiveAmount = 6000
WHERE IncentiveRefId IN (
    SELECT EmployeeId
    FROM Employee
    WHERE FirstName = 'John'
);
```

Qes71)

```
CREATE TABLE Employee (
    Employee_Id  NUMBER,
    First_Name   VARCHAR2(50),
    Last_Name    VARCHAR2(50),
    Salary       NUMBER,
    Joining_Date DATE,
    Department   VARCHAR2(50)
);
```

Qes72)

```
DROP TABLE Employee;
```

Qes73)

```
ALTER TABLE Employee
ADD CONSTRAINT Employee_PK
PRIMARY KEY (Employee_Id);
```

Qes74)

```
ALTER TABLE Employee
ADD CONSTRAINT Employee_PK
PRIMARY KEY (Employee_Id, First_Name);
```

Qes75)

```
ALTER TABLE Employee
DROP CONSTRAINT Employee_PK;
```

Qes76)

```
ALTER TABLE Incentives
ADD CONSTRAINT Incentives_Employee_FK
FOREIGN KEY (Employee_Ref_Id)
REFERENCES Employee (Employee_Id);
```

Qes77)

```
ALTER TABLE Incentives
DROP CONSTRAINT Incentives_Employee_FK;
```

Qes78)

```
CREATE SEQUENCE Employee_Seq
START WITH 1
INCREMENT BY 1;
```

Qes79)

```
CREATE OR REPLACE TRIGGER Employee_BI
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    :NEW.Employee_Id := Employee_Seq.NEXTVAL;
END;
/
```

Qes80)

SQL Injection is when a hacker puts **SQL code inside an input field** (like username or password) to **trick the database into doing something it shouldn't**.

Qes81)

A data base transaction consists of:

Group of SQL Operation (DML, DDL, DCL statements) that are treated as one unit of work

COMITT

When you are sure of your changes and want to save them, it makes changes permanent.

ROLLBACK

If you made changes and want to undo them **before they are committed**